

WAYPOINT R2

WAYPOINT

Review 2 Backend & System Architecture Guide

What the Two Backend Engineers Must Build, Connect, Test, and Present

Review 2 Date: Friday 9 October 2026 | Prepared Sunday 4 October 2026

The Goal in One Sentence: By Review 2, a student's profile is saved in SQLite, Python reads destinations and skills from CSV data, calculates skill gaps and 3 deterministic study routes, generates the DAG React Flow graph, serves authentic live opportunities, and both engineers can explain every calculation in viva without hesitation.

Updated for Current Repository: This guide strictly reflects your real frontend implementation (Next.js 14, 8 career destinations, 12 engineering branches, dynamic React Flow DAG generator, and authentic Opportunities & Radar ecosystem). Where older plan documents disagree, **this guide represents the truth**.

1. What the Backend Must Deliver for Review 2

Review 2 requires 5 core operational capabilities from the backend. Everything else (Gemini LLM explainers, student crowd trails, and market weather scraping) is scheduled for Review 3.

Step	What Happens	Technical Mechanism	Owner
1. Profile Storage	Student completes onboarding. Backend saves profile and returns it on next visit.	<code>POST /api/student/profile</code> (Header: <code>X-User-Id</code>). Saves into SQLite table <code>profiles</code> .	Person 3
2. Registry Data	Backend provides destinations, skills, prerequisites, and verified opportunities.	Reads <code>data/*.csv</code> via cached <code>data_loader.py</code> . <code>GET /api/destinations</code> .	Person 3
3. Skill Gap Engine	Computes exact gap numbers, prerequisite blockers, and priority labels.	<code>POST /api/skills/gap-analysis</code> via deterministic math in <code>skill_gap.py</code> .	Person 2
4. Route & DAG Builder	Generates Fast Track, Balanced, and Foundation routes with React Flow nodes/edges.	<code>POST /api/routes</code> and <code>POST /api/routes/{id}/graph</code> via <code>route_builder.py</code> .	Person 2
5. Opportunities Radar	Returns authentic hackathons, research fellowships, internships, and startup incubators.	<code>GET /api/opportunities</code> . Reads <code>data/opportunities.csv</code> with daily trending scores.	Person 3 & 2

End-to-End Execution Flow

Student Onboarding (Frontend)	
POST /api/student/profile (Header: X-User-Id)	→ Person 3 saves to SQLite DB
▼	
Choose Destination & View Details	
GET /api/destinations	→ Person 3 serves from destinations.csv
▼	
Skill Gaps Screen	
POST /api/skills/gap-analysis	→ Person 2: skill_gap.py (Mathematical Gaps)
▼	
Routes Overview Screen	
POST /api/routes	→ Person 2: route_builder.py (3 Timeline Schedules)
▼	
Interactive Route DAG Map	
POST /api/routes/{routeId}/graph	→ Person 2: Topological Graph (Nodes + Edges)
▼	
Opportunities & Action Radar	
GET /api/opportunities?destination=...	→ Person 3 & 2: Verified Govt & Industry Programs

Division of Responsibilities

Dimension	Person 2 — Engines (The Brain)	Person 3 — Data & Platform (The Memory)
In Simple Words	The algorithms: mathematics, prerequisite graph traversal, scheduling timelines, and DAG layout.	The storage: CSV data schemas, validation scripts, SQLite database, and web server wiring.
Owned Files	services/skill_gap.py services/route_builder.py schemas/skill.py , schemas/route.py routers/skills.py , routers/routes.py tests/test_skill_gap.py tests/test_route_builder.py	data/*.csv (5 canonical CSVs) scripts/validate_data.py services/data_loader.py core/database.py , core/config.py models/profile.py , schemas/profile.py routers/student.py , routers/destinations.py main.py , tests/test_data.py

2. Current Repository State & Baseline

Component / File	State Before	Required Review 2 Action
backend/main.py	Only / and /health . CORS set to allow_origins=["*"] with credentials.	Mount all 5 routers; configure CORS strictly for http://localhost:3000 .
data/skills.csv	Only 3 dummy rows with SKILL_01 IDs.	Populate full 68 skills using frontend slugs (python , sql , cryptography).
data/prerequisites.csv	Only 2 rows.	Populate 75 directed prerequisite dependencies across all tracks.
data/destinations.csv	Did not exist.	Create with all 8 real destinations matching frontend IDs.
data/destination_requirements.csv	Did not exist.	Create with 85 requirement mappings (skill_id, required_level, importance).
data/opportunities.csv	Old dummy data.	Populate with 18 verified government and international programs (SIH, Kavach, PMRF, GSoC).
requirements.txt	Missing sqlalchemy and httpx .	Added sqlalchemy>=2.0.0 and httpx>=0.27.0 .

3. Seven Shared Rules (Must Agree on Day 1)

Inconsistent naming is the #1 cause of bugs in split frontend/backend teams. Follow these rules strictly:

- Skill IDs are slugs:** Always use exact lowercase slugs like python , data-structures , machine-learning , cryptography . Never use arbitrary codes like SKILL_01 .
- Skill Levels are words:** None = 0 , Beginner = 1 , Intermediate = 2 , Advanced = 3 . Exactly match these string literals in CSVs and JSON responses.
- JSON is camelCase, Python is snake_case:** In Pydantic models, always configure populate_by_name = True and alias_generator = to_camel so frontend gets currentSkills , estimatedWeeks , totalWeeks .
- Engines are Stateless:** The skill gap and route endpoints are POST requests that receive the student's profile/skills and return calculations immediately. No database dependency inside engine logic.
- User is Identified by Header:** X-User-Id (e.g. user_1728...). Person 3 stores one persistent profile per user ID in SQLite.
- Default Level Assumption:** Listed skills in student onboarding default to Intermediate . The engine also accepts an optional skillLevels dictionary for granular overrides.
- Root Execution:** Always run from repo root: uvicorn backend.main:app --reload --port 8000 and pytest .

PERSON 3 — DATA & PLATFORM

4. Person 3: Data, Storage & Web Platform

Person 3 provides the knowledge base and persistence. If CSV data is corrupt or cyclic, Person 2's algorithms cannot function.

T3-1: The Five Canonical CSV Files in `data/`

File	Columns	Description & Constraints
<code>skills.csv</code>	<code>skill_id</code> , <code>skill_name</code> , <code>category</code> , <code>description</code> , <code>why_it_matters</code> , <code>estimated_weeks</code>	68 skills shared across all 8 destinations. Categories must belong to the 10 allowed categories.
<code>prerequisites.csv</code>	<code>skill_id</code> , <code>prerequisite_id</code> , <code>dependency_strength</code>	75 directed edges. States: " <code>skill_id</code> requires <code>prerequisite_id</code> first." Must be 100% cycle-free.
<code>destinations.csv</code>	<code>destination_id</code> , <code>title</code> , <code>description</code> , <code>icon</code> , <code>category</code> , <code>avg_salary</code> , <code>tags</code>	8 career tracks (<code>ai-ml-engineer</code> , <code>software-engineer</code> , <code>cybersecurity-engineer</code> , <code>mtech-gate</code> , etc.).
<code>destination_requirements.csv</code>	<code>destination_id</code> , <code>skill_id</code> , <code>required_level</code> , <code>importance</code>	85 requirement mappings. Levels: <code>Beginner</code> , <code>Intermediate</code> , <code>Advanced</code> . Importance: <code>core</code> , <code>supporting</code> , <code>optional</code> .
<code>opportunities.csv</code>	<code>opportunity_id</code> , <code>title</code> , <code>organization</code> , <code>type</code> , <code>description</code> , <code>deadline</code> , <code>stipend_or_prize</code> , <code>location</code> , <code>eligibility</code> , <code>url</code> , <code>mode</code> , <code>destinations</code> , <code>tags</code> , <code>featured</code> , <code>trending_score</code>	18 authentic, verified programs (SIH, Kavach, Amazon ML School, GSoC, DRDO, PMRF, DAAD, Hult Prize, NIDHI-EIR).

T3-2: `scripts/validate_data.py`

Person 3 must run this validation script before committing any dataset modification. It automatically asserts:

- Foreign key consistency: Every `skill_id` and `prerequisite_id` exists in `skills.csv` .
- Every destination in `destination_requirements.csv` exists in `destinations.csv` .
- Acyclicity check:** Depth-first search (DFS) confirms no prerequisite cycles exist ($A \rightarrow B \rightarrow A$).
- Closure Rule:** If a destination requires a skill, it must also include all transitive prerequisites of that skill.
- Minimum requirement count: Every destination must define at least 5 skills.

T3-3: `services/data_loader.py`

The single source of truth for reading CSVs in Python. Person 2 never opens CSVs directly. All loaders use `@functools.lru_cache` for instant in-memory responses:

```
# services/data_loader.py interface:
get_skill(skill_id: str) -> dict
list_skills() -> list[dict]
get_prerequisites(skill_id: str) -> list[str]
get_dependents(skill_id: str) -> list[str]
list_destinations() -> list[dict]
get_destination(destination_id: str) -> dict | None
get_requirements(destination_id: str) -> list[dict]
list_opportunities(destination_id: str = None, opp_type: str = None) -> list[dict]
```

T3-4: SQLite Database & Profile Storage

- `core/database.py` : Configures `create_engine("sqlite:///./waypoint.db")` , `SessionLocal` , `Base` , and `get_db()` generator.
- `models/profile.py` : Table `profiles` storing: `user_id` (String, PK), `name` (String), `program` (String), `branch` (String), `semester` (Integer), `cgpa` (Float), `credits_completed` (Integer), `current_skills` (JSON string/list), `selected_destination_id` (String), `updated_at` (DateTime).
- Tables are automatically initialized on startup via `Base.metadata.create_all(bind=engine)` in `main.py` .

PERSON 2 — ENGINES

5. Person 2: Deterministic Algorithmic Engines

Person 2 builds the core algorithmic intelligence that examiners will test during viva. **No black-box LLMs or non-deterministic ML models are used here** — all output is pure, verifiable graph theory and mathematical scheduling.

T2-1: `services/skill_gap.py` — The Gap Analyzer Engine

The gap calculation algorithm operates in 8 deterministic steps:

- Current Level Evaluation:** For each required destination skill, if present in student's `current_skills`, assign level `Intermediate (2)` (or read from `skill_levels` map). If not present, assign `None (0)`.
- Gap Value:** $\text{gap} = \text{level}_{\text{required}} - \text{level}_{\text{current}}$. If $\text{gap} \leq 0$, the skill is satisfied.
- Gap Type:** If $\text{level}_{\text{current}} == \text{None}$ $\implies$ `'full'`; if $\text{level}_{\text{current}} > \text{None}$ $\implies$ `'partial'`.
- Prerequisite Blockers (`blocks_count`):** Count how many other unsatisfied gap skills list this skill as a direct prerequisite.
- Priority Assignment:**
 - If $\text{blocks_count} \geq 2$ $\implies$ `Critical` (major roadblock).
 - Else if $\text{importance} == \text{'core'}$ $\implies$ `High` Priority.
 - Else $\implies$ `Recommended`.
- Status for DAG Map:**
 - `completed`: student already has the skill.
 - `next`: student needs the skill, and *all* its prerequisites are completed.
 - `locked`: student needs the skill, but one or more prerequisites are still missing.
- Sorting:** Sort gaps primarily by Priority (`Critical` to `High` to `Recommended`), secondarily by gap magnitude, and tertiarily by title.
- Audit Explanation:** Fill a deterministic sentence template with real numbers (e.g. "Required Advanced, currently at None (gap 3). Blocks 2 downstream skills you need.").

Worked Verification Example: Student Alex (AI/ML Engineer)

Alex has `["python", "cpp", "html-css", "git"]`. The engine must return exactly these values:

Skill Name	Required Level	Gap	Type	Blocks	Priority	Status
machine-learning	Advanced (3)	3	full	2	Critical	locked
data-structures	Intermediate (2)	2	full	2	Critical	next
deep-learning	Intermediate (2)	2	full	2	Critical	locked
linear-algebra	Intermediate (2)	2	full	3	Critical	next
statistics	Intermediate (2)	2	full	2	Critical	locked
data-analysis	Intermediate (2)	2	full	1	High	locked
sql	Intermediate (2)	2	full	1	High	locked
mlops	Beginner (1)	1	full	0	Recommended	locked
cloud-basics	Beginner (1)	1	full	0	Recommended	next (needs git)

T2-2: `services/route_builder.py` — **Multi-Track Timeline Scheduler**

A route is an ordered weekly roadmap. The engine builds three distinctive variants:

Route ID	Route Name	Workload	Concurrent Slots	Pace Factor	Skills Included
<code>fast-track</code>	Fast Track	High	3 skills at once	0.7× duration	Critical + High priority gaps only.
<code>balanced</code>	Balanced Route	Medium	2 skills at once	1.0× duration	Critical + High + Supporting skills (<code>isRecommended: true</code>).
<code>foundation-first</code>	Foundation First	Low	2 skills at once	1.3× duration	All gap skills (includes Optional + foundational deep-dives).

Scheduling Algorithm:

- $\text{weeks} = \max(1, \text{round}(\text{estimated_weeks} \times \text{pace_factor}))$.
- A skill is eligible to begin in week W only after *all* its prerequisite skills have completed ($\text{weekEnd} < W$).
- Available study slots are prioritized by Priority (Critical first, then larger gaps).
- `totalWeeks` is the maximum `weekEnd` across all steps; `skillCount` is the total skills in the route.

T2-3: Topological React Flow DAG Generator

Generates the node and edge structures required by the frontend's React Flow canvas:

- **Nodes:** Exactly one `start` node at $(340, 20)$, completed prerequisites, active gap skills, and one final `goal` node.
- **Topological Layering:** Compute the longest prerequisite path from start to each node: $\text{layer}(S) = \max_{\{\text{prereqs } P\}} (\text{layer}(P)) + 1$ $\text{coordinate } y = 20 + \text{layer} \times 110$
- **Horizontal Spacing:** In each layer, distribute N nodes symmetrically centered around $x = 340$, spaced 180px apart.
- **Edges:** Connect `prerequisite_id` to `skill_id` with `type: 'smoothstep'`. Connect `start` to skills with no prerequisites; connect terminal skills to `goal`. Set `animated: true` for edges leading into `next` skills.

6. Complete API Cheat Sheet (Deliver to Person 1)

Endpoint & Method	Owner	Request Headers / Body	Response Shape
POST /api/student/profile	P3	Header: X-User-Id Body: StudentProfile	Returns saved StudentProfile JSON
GET /api/student/profile	P3	Header: X-User-Id	Returns StudentProfile or 404 Not Found
GET /api/destinations	P3	None	Returns list[Destination]
GET /api/opportunities	P3	Query: destination, type, filter	Returns list[Opportunity] (with daily trending scores)
POST /api/skills/gap-analysis	P2	Body: { destinationId, currentSkills, skillLevels? }	{ skills: list[Skill], gaps: list[SkillGap] }
POST /api/routes	P2	Body: { destinationId, currentSkills, skillLevels? }	Returns list[Route] (Fast Track, Balanced, Foundation)
POST /api/routes/{id}/graph	P2	Body: { destinationId, currentSkills, skillLevels? }	{ nodes: list[Node], edges: list[Edge] }

Sample JSON Payloads

```
// 1. Request Body for Engine Endpoints (Gap Analysis, Routes, Graph):
{
  "destinationId": "ai-ml-engineer",
  "currentSkills": ["python", "cpp", "html-css", "git"],
  "skillLevels": { "python": "Intermediate" }
}

// 2. Sample Route Object (POST /api/routes):
{
  "id": "fast-track",
  "name": "Fast Track",
  "tagline": "Move fast. Cover the critical essentials.",
  "description": "High-intensity accelerated plan covering critical path skills.",
  "workload": "High",
  "totalWeeks": 16,
  "skillCount": 7,
  "isRecommended": false,
  "steps": [
    { "skillId": "data-structures", "weekStart": 1, "weekEnd": 2 },
    { "skillId": "linear-algebra", "weekStart": 1, "weekEnd": 3 }
  ]
}

// 3. Sample Graph Node & Edge (POST /api/routes/{routeId}/graph):
{
  "nodes": [
    {
      "id": "data-structures",
      "type": "skillNode",
      "position": { "x": 250, "y": 130 },
      "data": { "skillId": "data-structures", "label": "Data Structures", "category": "Foundations", "status": "next", "priority": "critical" }
    }
  ],
  "edges": [
    { "id": "e-python-data-structures", "source": "python", "target": "data-structures", "type": "smoothstep", "animated": true }
  ]
}
```

7. Day-by-Day Implementation Plan (Sun 4 Oct – Fri 9 Oct)

Day	Person 3 — Data & Platform	Person 2 — Engines
Sun 4 Oct	Generate canonical CSVs using <code>generate_seed_csvs.py</code> . Run <code>validate_data.py</code> . Update <code>requirements.txt</code> .	Write Pydantic schemas (<code>schemas/skill.py</code> , <code>schemas/route.py</code>). Build stub endpoints returning test JSON.
Mon 5 Oct	SQLite database setup, <code>models/profile.py</code> , profile router, <code>main.py</code> CORS wiring for <code>http://localhost:3000</code> .	Implement <code>skill_gap.py</code> with 8-step logic. Verify Alex's worked example test passes.
Tue 6 Oct	Write <code>services/data_loader.py</code> with LRU cache. Add <code>routers/destinations.py</code> and <code>routers/opportunities.py</code> .	Implement <code>route_builder.py</code> timeline scheduling algorithm and unit tests.
Wed 7 Oct	Write API integration tests (<code>tests/test_profile_api.py</code> , <code>tests/test_data.py</code>).	Build React Flow graph coordinate layout generator. Replace stubs with live engines.
Thu 8 Oct	Integration Day: Run frontend against live FastAPI server. Practice viva questions together.	Integration Day: Verify graph renders smoothly in frontend. Fix edge-case scheduling bugs.
Fri 9 Oct	REVIEW 2 DEMO & VIVA PRESENTATION	REVIEW 2 DEMO & VIVA PRESENTATION

8. Done Checklist for Review 2

- ☐ `python scripts/validate_data.py` exits with status 0 (no cycles, closure satisfied).
- ☐ `pytest` runs from repo root and all tests pass (0 failures).
- ☐ Saving a profile via `POST /api/student/profile` survives server restart in SQLite.
- ☐ `POST /api/skills/gap-analysis` accurately calculates gaps, blockers, and priorities.
- ☐ `POST /api/routes` generates 3 routes with valid weekly prerequisite orderings.
- ☐ `POST /api/routes/{routeId}/graph` outputs valid React Flow nodes and edges.
- ☐ `GET /api/opportunities` returns verified programs with daily rotating badges.
- ☐ Both engineers can open their own code and answer examiner questions without notes.

9. Viva Cheat Sheet — Say It in Plain Words

Examiner Question	Recommended Engineering Answer
"Why not use an LLM (like GPT-4) to generate routes?"	LLMs hallucinate prerequisite orders, are non-deterministic, cost per token, and take 3–5 seconds per call. Our topological scheduler is deterministic, instantaneous (<10ms), 100% mathematically verifiable, and works offline without API costs.
"How do you determine which skills are Critical vs High Priority?"	We calculate a <code>blocks_count</code> : how many other missing skills have this skill as a direct dependency. If a skill blocks 2 or more downstream skills, it is mathematically Critical because it bottlenecks the entire journey.
"How does the route scheduler assign weeks?"	It uses a constrained bin-packing model: a skill can only start in week W if all its prerequisites finished in week $W-1$, and subject to maximum concurrent study slots (2 for Balanced, 3 for Fast Track).
"How is data scalability handled when adding new careers?"	The engine is entirely data-driven. To add a new career track (e.g. Cloud Architect), we simply add rows to <code>destinations.csv</code> and <code>destination_requirements.csv</code> . Zero lines of Python code need to be rewritten.
"How do you ensure authenticity of opportunities?"	Every entry in <code>data/opportunities.csv</code> links directly to official government bodies (AICTE, Ministry of Defence iDEX, MoE PMRF) or global programs (Google Summer of Code, Y Combinator, DAAD) with verified URLs.